
UrbanFlowRL – RL for Optimizing Subway Flow in NYC

Fessa Soenanta Veronica Yakubovich

Project Mentor: Alex Nam

Abstract

Public transportation plays a critical role in urban mobility, yet traditional subway scheduling methods struggle to adapt to real-time passenger demand, leading to inefficiencies such as overcrowding and long wait times. In this project, we present UrbanFlowRL, a reinforcement learning-based approach to optimizing train assignments in the New York City subway system. Our method utilizes historical ridership data and a custom Gymnasium environment to train reinforcement learning agents that dynamically adjust train schedules to minimize waiting times and balance passenger loads. We evaluate three reinforcement learning algorithms: Advantage Actor-Critic (A2C), Proximal Policy Optimization (PPO), and Deep Q-Networks (DQN), against a fixed scheduling baseline derived from MTA data. Our findings highlight the potential of policy-based RL methods in optimizing subway scheduling. Future work includes incorporating real-time disruptions, adaptive reward functions, and additional operational constraints to enhance real-world applicability.

1. Introduction

Public transportation has the potential to be a green and affordable solution to the social cost of mobility challenges, offering a cost-effective and environmentally-friendly alternative to cars. The New York City (NYC) subway system is one of the largest transit systems in the world, and millions of New Yorkers use the NYC subway system every day to get around the city. However, frequent overcrowding, unpredictable wait times, and inefficient train scheduling contribute to commuter frustrations, and subway users may be deterred by slow or unreliable systems. Addressing these issues is key to developing more accessible and sustainable transit systems.

Traditional scheduling methods rely on fixed timetables and periodic adjustments based on historical demand patterns. However, these approaches often fail to adapt to real-time fluctuations in ridership, leading to uneven train loads, excessive wait times, and inefficient use of available resources.

A more adaptive scheduling system that dynamically adjusts train dispatch frequencies in response to real-time demand could significantly enhance subway efficiency and the commuter experience.

In this project, we propose UrbanFlowRL: a reinforcement learning based approach to optimizing subway scheduling in NYC. By leveraging historical ridership data, our method aims to develop more effective policies for dispatching trains in the NYC subway system. Reinforcement learning provides an approach to optimizing sequential decision-making problems and is an effective tool for dynamically adjusting train schedules while simultaneously minimizing wait times and overcrowding and ensuring operational feasibility.

To achieve this, we utilize data from the NYC Metropolitan Transit Authority (MTA) data catalog ([Metropolitan Transportation Authority, 2023](#)), where we have access to ridership statistics, station entries, and historical train schedules. These datasets serve as the foundation for training our RL agent, which learns scheduling policies that optimize train frequency to minimize wait time and overcrowding. By developing an adaptive scheduling framework, we aim to create a model that responds to fluctuating demand and improves the efficiency of the NYC subway system.

2. Related Work

Reinforcement learning (RL) has been extensively applied to public transportation and logistics optimization, demonstrating its ability to improve scheduling efficiency, energy consumption, and passenger experience. Existing literature highlights RL-based approaches for optimizing metro train scheduling, bus network design, and train trajectory optimization.

For bus network design, deep learning has been applied to transportation and logistics, and there has been work utilizing DQNs to optimize bus timetables based on real-time demand in order to improve service by minimizing wait times ([Ai et al., 2021](#)). RL has also been applied to improve bus network design by determining optimal routes, route frequency, and number of routes needed ([Yoo et al., 2023](#)).

For subway and metro train scheduling, Ying et al. (2020) proposed an actor-critic deep RL approach to optimize real-

time metro train scheduling under stochastic demand, utilizing Deep Deterministic Policy Gradient (DDPG) to efficiently allocate rolling stock in the London Underground Victoria Line. Their study showed that actor-critic agents outperformed fixed schedules, demonstrating the potential of RL in managing real-world subway operations (Ying et al., 2020).

Another key area of RL application in transportation is energy-efficient train operation. Xie et al. (2023) explored the use of Proximal Policy Optimization (PPO) to optimize metro energy efficiency under uncertain conditions. Their model adjusted cruising speed and dwell time to minimize energy consumption and achieved a 47.9% increase in regenerative braking energy utilization using real-world data from Xiamen Line 1 (Xie et al., 2023).

Beyond scheduling, RL has also been used for train trajectory optimization. Cheng et al. (2024) developed a train trajectory optimization (TTO) model that discretizes train movement into a Markov Decision Process (MDP). Their approach incorporated safety reinforcement learning with dynamic speed limit constraints, improving operational efficiency and passenger safety (Cheng et al., 2024).

Additionally, Zhou et al. (2021) introduced Smart Train Operation Algorithms (STOD and STON), which integrate expert driver knowledge with RL models to optimize subway operations. Their approach demonstrated energy efficiency improvements while ensuring real-time adaptability (Zhou et al., 2020).

Our work, UrbanFlowRL, builds upon these prior studies by applying RL-based scheduling optimization to large-scale subway networks. Unlike existing approaches that focus on individual train efficiency or bus networks, our model dynamically assigns train frequencies across multiple subway lines in response to fluctuating demand. By leveraging historical ridership data and real-time adjustments, UrbanFlowRL aims to reduce wait times, balance passenger loads, and improve overall subway efficiency.

3. Approach

3.1. Data Pre-processing

To analyze NYC subway ridership, we obtained hourly ridership data from the NYC MTA data catalog (Metropolitan Transportation Authority, 2023). This dataset contains hourly ridership for each subway station that is part of the NYC subway system. Historically, there are two types of subway tracks - the IRT and BMT/IND lines - which were built separately and have different track dimensions and cannot share subway cars. We chose to focus on the 13 lettered lines (Q, F, D, etc), which are otherwise known as the BMT and IND lines. This gives a total of 382 trains to

be shared among the 13 lines, based on MTA’s latest estimation (Metropolitan Transportation Authority, 2024). In addition, we used the hourly data from 2024 and aggregated into five time categories per day per line: AM Peak (6-9 AM), Midday (9 AM-3 PM), PM Peak (3-7 PM), Evening (7-11 PM), and Overnight (11 PM-6 AM), following MTA definitions. We excluded IRT trains using station complex identifiers from MTA subway station datasets and assumed that ridership at stations servicing multiple lines was evenly distributed among lines. Additionally, we calculated average travel times between stations by dividing the total runtime by the number of stops. The dataset also includes information on the order of stops per line, as well as the number of trains assigned to each line on Weekdays, Saturdays, and Sundays per the five time categories defined above, allowing us to estimate average travel times across entire lines.

3.2. Environment

To evaluate our models, we created a custom environment using OpenAI’s gymnasium (Towers et al., 2024).

Observation Space

The observation space is a

```
Dict({"line":Box(0,13,(382,),int8),
      "location":Box(0,60,(382,),int8),
      "direction":MultiBinary(382)})
```

This represents the line that each train is at, its location along the line as well as the direction it is facing. Each of the 382 trains can be in one of 13 lines and be anywhere along it, with the longest line containing 60 stops.

Action Space

The action space is a

```
MultiDiscrete(np.repeat(13,382))
```

An action represents assigning one out of the 13 possible lines to each of the 382 trains.

Reward Function

The reward aims to penalize passenger overflow and reward efficient boarding. While constructing the reward function, we encountered issues with exploding rewards that led to unstable training. To counteract this, we implemented a clipping function and normalized the rewards to get a final function as follows:

$$reward = \frac{1}{1000} \sum_t \left[-\frac{\text{clip}(p, 0, 1e6) \cdot w \cdot d}{1e6} + \mathbb{1}_{\{b \geq 0.8\}} b \right]$$

where p is the number of leftover passengers at the stations, w is the time it takes for the train to get to the next sta-

tion (representing wait time), d is a demand variable that fluctuates depending on the time of day, and b is a bonus given for efficient boarding. The reward is calculated for each timestep t of the day, then summed to get the total rewards for the action. The demand d ranges from 0.5 to 1.5, giving more weight to the penalty during peak times and less weight during the overnight time period. The bonus b is the average train occupancy, and is only rewarded when the trains are over 80% full.

Episode End

The environment will terminate if the the time between trains is below 2 minutes, as we consider this too short of an interval and would be dangerous in real life. The environment truncates once it has taken 365 steps, which represents the year of data we have.

Framework

At each step, the environment simulates the movements of the trains for a day. Each train visits stations of the line it is on in order, with trains being evenly spaced along the line. When a train reaches the end of a line, all of its passengers alight and it 'switches' direction, now visiting stations in the opposite order. After each day, the number of passengers at stations and trains are reset to 0.

At each timestep, the stations are populated with passengers according to the real-world data, which fluctuates depending on the day and time period. Then, some number of passengers currently on board (chosen uniformly random) will alight and the train moves to the next station. Lastly, the maximum number of passengers from the station will board, based on the available space on the train. Figure 1 illustrates the passenger flow for a train at each timestep, using sample numbers and stations.

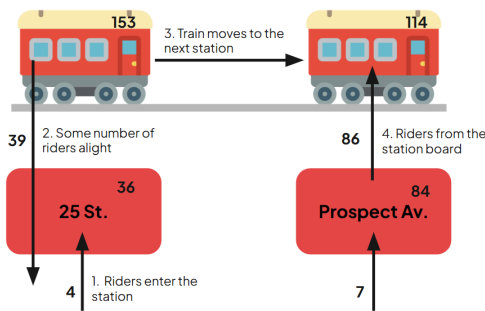


Figure 1. Environment Visualization

In the toy example, the riders that enter the stations (4 for 25 St. and 7 for Prospect Av.) is calculated from the data and is added to the number of riders already at the station (36 and 84 for 25 St. and Prospect Av. respectively). The number of passengers that got off at 25 St. (39) is chosen at random from the number of passengers currently in the

train (153) and then subtracted, which causes the train to have 114 passengers when it arrives at Prospect Av. Finally, the train takes in as many passengers as possible, which in this case is only 86 out of the available 91 since the train caps at 200 riders. The number of passengers that fail to board the train is used to calculate rewards.

When the agent takes an action, it will assign each train a line. If the assignment stays the same, the train continues to move as before. Else, the train is moved to the appropriate line and the trains in the line are adjusted so they are still evenly spaced. This moving and adjusting each takes half an hour, so the trains do not start moving until an hour into the day. This is meant to simulate the delay it takes to move trains between lines.

3.3. Methods

To evaluate the performance of our RL-based scheduling method, we compare it to a fixed scheduling approach where trains are assigned to lines based on historical data. Our fixed schedule is derived from real-world train frequency data provided by the NYC MTA.

Baseline

The baseline schedule follows a static allocation of trains to each subway line, determined through historical scheduling patterns. We used the End-to-End Running Times dataset from the MTA, which records actual train assignments across different subway lines and time periods. To estimate the daily distribution of trains across the subway system, we first aggregated the number of trains running on each line during different time periods of the day. Next, we computed the average train count for each line across all available time periods within a given month. To further smooth fluctuations, we averaged these monthly train counts across multiple months, yielding a representative daily train count for each subway line. After obtaining these daily estimates, we filtered the data to retain only the BMT and IND lettered subway lines, as our model does not include IRT lines. Since the total number of available trains in the system is fixed at 382, we normalized the train counts so that their sum matched this constraint. Using these counts, we can construct an action that assigns an appropriate number of trains to each line. This action is and passed through the environment regardless of the current observation until the episode ends to get total rewards. This was repeated for 20 episodes to serve as our baseline model.

Advantage Actor-Critic

Actor-Critic is a reinforcement learning algorithm that combines policy-based and value-based methods. It consists of an actor that updates the policy and a critic that evaluates the action by estimating its value. At each timestep t , the current state s_t is passed through the actor, a policy function $\pi_\theta(a|s)$, to output an action a_t . Then the critic, a value

function $q_w(s, a)$, takes the state and action and computes the Q-value. The action is performed to get a new state s_{t+1} and a reward r_t . The actor updates its parameters using the Q-value

$$\Delta\theta = \alpha \nabla_{\theta} (\log \pi_{\theta}(a|s)) q_w(s, a)$$

and produces the next action a_{t+1} to take at the new state. The critic then updates its parameters

$$\Delta w = \beta(r_t + \gamma q_w(s_{t+1}, a_{t+1}) - q_w(s_t, a_t)) \nabla_w q_w(s_t, a_t)$$

Advantage Actor-Critic (A2C) improves upon this by using the advantage function as the critic:

$$A(s, a) = Q(s, a) - V(s)$$

where $Q(s, a)$ is the expected value after taking action a from state s and $V(s)$ is the expected value from state s . Essentially, the advantage function calculates how much better taking that action at that state is compared to the average value of the state. When updating the policy, an estimate of the advantage function is used:

$$A(s_t, a_t) = r_t + \gamma V(s_{t+1}) - V(s_t)$$

The use of both the actor and critic networks result in more stable learning. Furthermore, by leveraging the advantage function, the A2C model is able to reduce gradient variance that leads to more efficient training. A2C is also more cost-effective than Asynchronous Advantage Actor-Critic (A3C) (Mnih et al., 2016), the algorithm it was originally implemented from. A2C is the synchronous, deterministic variant of A3C that has been found to perform better with faster training and more effective use of GPUs, especially with larger policies.

We utilized the stable-baselines3 library (Raffin et al., 2021) to implement this algorithm, which allows for the multiple inputs needed for our defined action space. Using the default parameters of 0.0007 learning rate and 0.99 discount factor, we trained for 7300 steps, which is equivalent to 20 episodes if the environment does not terminate early.

Proximal Policy Optimization

Policy gradient is a common RL algorithm that work by updating the policy using gradient descent. However, this method often leads to exploding updates steps. Proximal Policy Optimization (PPO) mitigates this issue by utilizing a clipped objective (Schulman et al., 2017):

$$L^{CLIP}(\theta) = \mathbb{E}[\min(r_t(\theta)) \hat{A}_t, \text{clip}(r_t(\theta), 1-\epsilon, 1+\epsilon) \hat{A}_t)]$$

where $r_t = \frac{\pi_{\theta}(a_t|s_t)}{\pi_{\theta_{\text{old}}}(a_t|s_t)}$ and $\hat{A}_t$ is the estimated advantage.

The clipping mechanism avoids drastic updates, leading to more stable training compared to vanilla policy gradient.

It is also more sample efficient, using mini-batch updates that allows for faster learning. Furthermore, it is still relatively easy to implement and tune compared to methods like TRPO. This combination results in a robust method that performs well on various tasks. Similar to A2C, we also utilized the stable-baselines3 library and used the default parameters to train for 7300 steps.

Deep Q-Learning

Deep Q-Networks (DQN) is a value-based RL algorithm that approximates Q-values using a neural network. In our approach, we implement a DQN agent to optimize subway train scheduling by dynamically assigning trains to lines based on observed states.

The DQN agent consists of a neural network that maps state observations to Q-values for each possible action. Our implementation uses a feedforward network with two hidden layers of 256 units each and ReLU activations. The network is trained using the AdamW optimizer with a learning rate of $1e^{-3}$. The agent maintains an ϵ -greedy exploration strategy, where ϵ decays exponentially from 1.0 to 0.01 over the training process to balance exploration and exploitation.

The training process follows the standard Q-learning update rule, where the target Q-value is computed as:

$$Q(s, a) \leftarrow r + \gamma \max_{a'} Q(s', a'),$$

where r is the observed reward, $\gamma = 0.99$ is the discount factor, and s' is the next state.

To stabilize training, we use a target network that is updated periodically to reduce correlations in Q-value updates. The agent also maintains a replay buffer of size 50,000 to sample uncorrelated experiences during training. The training process consists of drawing mini-batches of size 64 from the replay buffer and performing gradient descent to minimize the mean squared error (MSE) loss between the predicted and target Q-values.

In each episode, the agent starts with an initial state and iteratively selects actions, receives rewards, and transitions to new states until termination. The reward function penalizes long wait times and passenger overflow while incentivizing efficient train assignments. The agent updates the policy based on collected experiences and periodically synchronizes the target network with the main network.

4. Results

To evaluate the effectiveness of our RL models, we compared the performance of Advantage Actor-Critic (A2C), Proximal Policy Optimization (PPO), and Deep Q-Networks (DQN) against a baseline policy with a fixed schedule. Our evaluation metrics focus on total reward, peak reward, and average reward over the final five epochs.

4.1. Performance Comparison

Table 1 below shows the peak reward and average rewards over the last five epochs for each model:

Model	Peak Rewards	Average Rewards (over last 5 episodes)
Baseline	45.25	41.28
A2C	106.93	106.44
PPO	73.25	70.96
DQN	89.50	34.40

Table 1. Performance Comparison Across Models

From the results, A2C is clearly the best model of the three, achieving the highest overall rewards. PPO achieved decent peak and average rewards that beat the baseline, but still considerably lower than A2C. Lastly, DQN, while achieving a respectable peak reward, had lower average rewards than the baseline, indicating a large drop in performance.

Advantage Actor-Critic

A2C outperforms the other models and the baseline, achieving both the highest peak and average rewards. The addition of a critic network helps reduce variance, which leads to more stable learning. Additionally, A2C’s advantage function allows the model to make more informed and efficient updates, which contributes to its superior performance.

Proximal Policy Optimization

PPO performs moderately well, but does not achieve the same level of rewards as A2C. The clipping mechanism stabilizes learning and prevents large updates, leading to smaller but more consistent improvements. However, PPO is highly sensitive to hyperparameters, which may require additional fine-tuning for an improved performance. In this instance, the clipping parameter may be causing the updates to be too small and preventing the model from achieving high rewards like A2C. Tuning the hyperparameters could result in improved performance.

Deep Q-Learning

Although DQN offers a simple, yet effective approach to train allocation, it struggles with off-policy learning instability and reward sparsity. Noisy Q-value updates and the high-dimensional action space make convergence challenging, leading to suboptimal performance compared to policy-based methods like A2C and PPO.

4.2. Reward Trends Over Time

Figure 2 below shows the reward progression across 20 epochs for each RL model and the baseline:

A2C demonstrates consistent improvements and quickly converges to a high reward. PPO, while relatively stable, shows slower convergence. Furthermore, even though it ran

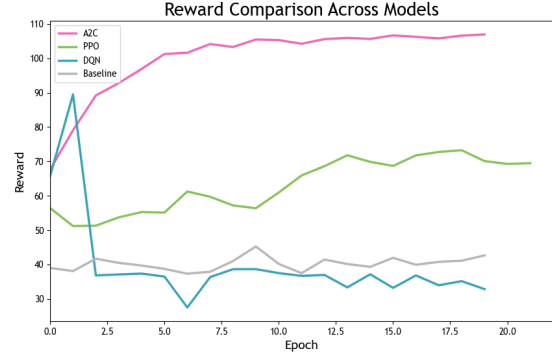


Figure 2. Rewards Across Models

for the same number of steps as A2C, it completed more episodes, indicating that some of the episodes terminated early. DQN exhibits large fluctuations, indicating difficulties in optimizing its Q-values effectively. In addition, it performed worse than the baseline.

The results demonstrate that policy-based methods (A2C, PPO) outperform value-based methods (DQN) in this train scheduling problem. The high-dimensional action space makes discrete-action Q-learning methods less effective, as they struggle with the large number of train assignment possibilities.

Future work could explore hybrid approaches, such as combining Q-learning with function approximation techniques or incorporating additional constraints to improve model generalization.

5. Conclusion

In this project, we present UrbanFlowRL, a reinforcement learning-based (RL) approach to optimizing subway scheduling in NYC. The NYC subway system faces persistent challenges such as overcrowding, unpredictable wait times, and inefficient train scheduling. Traditional fixed timetables struggle to adapt to real-time demand fluctuations, leading to inefficient resource allocation and passenger delays. Our approach leverages RL to dynamically assign trains across subway lines, minimizing wait times and balancing passenger loads.

We designed a custom Gymnasium environment that simulates subway train movements based on historical ridership data. The agent’s observation space includes train assignments, locations, and travel directions, while the action space involves reassigning trains to different lines. The reward function penalizes long wait times while incentivizing efficient scheduling.

We evaluated three RL models: Advantage Actor-Critic

(A2C), Proximal Policy Optimization (PPO), and Deep Q-Networks (DQN), against a fixed scheduling baseline derived from historical MTA train assignments. A2C outperformed all models, achieving the highest peak and average rewards due to its effective balance of exploration and exploitation. PPO performed moderately well but was sensitive to hyperparameters, while DQN struggled with off-policy learning instability and high-dimensional action spaces.

Our results demonstrate that policy-based methods are more effective than value-based methods for subway scheduling. By dynamically adjusting train allocations, reinforcement learning offers a promising solution to improving subway efficiency, accessibility, and sustainability.

While UrbanFlowRL has demonstrated the potential of reinforcement learning for optimizing subway scheduling, there are several directions for future improvement and exploration:

- *More Accurate Data from MTA:* Incorporating detailed data on train availability, rolling stock types, and actual transit schedules from the MTA will improve model accuracy and better reflect real-world constraints.
- *More Realistic Environment:* The current environment assumes ideal train operations, but real-world subway dynamics involve unexpected disruptions. Future work can introduce factors such as train breakdowns, maintenance schedules, and train rerouting to simulate real-world subway behavior more accurately.
- *Incorporate Additional Resource Constraints:* Beyond train scheduling, subway systems are constrained by train crew availability, power consumption, and track maintenance requirements. Incorporating these constraints would create a more holistic and feasible optimization framework.
- *Adaptive Reward Functions:* Different objectives may require tailored reward functions. Instead of a singular optimization criterion, future extensions could implement energy-efficient scheduling, congestion reduction, or balancing load distribution based on varying operational goals.
- *Multi-Armed Bandit Formulation:* Instead of treating subway scheduling as a sequential decision-making problem, an alternative approach could frame it as a multi-armed bandit problem, focusing on adaptive decision-making under uncertainty with faster learning dynamics.
- *Improved Generalization Across Transit Systems:* While this study focuses on NYC, future work could generalize the RL framework to other subway systems,

testing its adaptability to different transit networks with varying constraints.

By addressing these aspects, UrbanFlowRL can evolve into a more comprehensive and practically viable subway scheduling tool, capable of improving efficiency, reducing delays, and enhancing passenger experiences in real-world transit systems.

6. Contributions

Team members contributed equally towards the project. Fessa created most of the environment and implemented A2C and PPO while Veronica did the literature review, data pre-processing and DQN architecture.

References

- Ai, G., Zuo, X., Chen, G., and Wu, B. Deep reinforcement learning based dynamic optimization of bus timetable. *CoRR*, abs/2107.07066, 2021. URL <https://arxiv.org/abs/2107.07066>.
- Cheng, L., Cao, J., Yang, X., Wang, W., and Zhou, Z. A train trajectory optimization method based on the safety reinforcement learning with a relaxed dynamic reward. *Discover Applied Sciences*, 6(469), 2024. URL <https://doi.org/10.1007/s42452-024-06159-8>.
- Metropolitan Transportation Authority. Mta open data catalog and publication schedule, 2023. URL <https://data.ny.gov/d/f462-ka72>. Last accessed 26 February 2025.
- Metropolitan Transportation Authority. 2025-2029 capital plan: The future rides with us, 2024. URL <https://future.mta.info/capitalplan/>.
- Mnih, V., Badia, A. P., Mirza, M., Graves, A., Lillicrap, T. P., Harley, T., Silver, D., and Kavukcuoglu, K. Asynchronous methods for deep reinforcement learning. *CoRR*, abs/1602.01783, 2016. URL <http://arxiv.org/abs/1602.01783>.
- Paszke, A., Gross, S., Massa, F., Lerer, A., Bradbury, J., Chanan, G., Killeen, T., Lin, Z., Gimelshein, N., Antiga, L., Desmaison, A., Köpf, A., Yang, E. Z., DeVito, Z., Raison, M., Tejani, A., Chilamkurthy, S., Steiner, B., Fang, L., Bai, J., and Chintala, S. Pytorch: An imperative style, high-performance deep learning library. *CoRR*, abs/1912.01703, 2019. URL <http://arxiv.org/abs/1912.01703>.
- Raffin, A., Hill, A., Gleave, A., Kanervisto, A., Ernestus, M., and Dormann, N. Stable-baselines3: Reliable reinforcement learning implementations. *Journal of Machine*

- Learning Research*, 22(268):1–8, 2021. URL <http://jmlr.org/papers/v22/20-1364.html>.
- Schulman, J., Wolski, F., Dhariwal, P., Radford, A., and Klimov, O. Proximal policy optimization algorithms. *CoRR*, abs/1707.06347, 2017. URL <http://arxiv.org/abs/1707.06347>.
- Towers, M., Kwiatkowski, A., Terry, J., Balis, J. U., De Cola, G., Deleu, T., Goulão, M., Kallinteris, A., Krimmel, M., KG, A., et al. Gymnasium: A standard interface for reinforcement learning environments. *arXiv preprint arXiv:2407.17032*, 2024. URL <https://arxiv.org/abs/2407.17032>.
- Xie, H., Wang, C., Li, S., Zhang, Y., and Wang, S. Optimizing energy efficiency in metro systems under uncertainty disturbances using reinforcement learning. *arXiv*, 2023. URL <https://arxiv.org/abs/2304.13443>.
- Ying, C.-S., Chow, A. H., and Chin, K.-S. An actor-critic deep reinforcement learning approach for metro train scheduling with rolling stock circulation under stochastic demand. *Transportation Research Part B: Methodological*, 140:210–235, 2020. ISSN 0191-2615. URL <https://doi.org/10.1016/j.trb.2020.08.005>.
- Yoo, S., Lee, J. B., and Han, H. A reinforcement learning approach for bus network design and frequency setting optimisation. *Public Transport*, 15:503–504, 2023. URL <https://doi.org/10.1007/s12469-022-00319-y>.
- Zhou, R., Song, S., Xue, A., You, K., and Wu, H. Smart train operation algorithms based on expert knowledge and reinforcement learning. *CoRR*, abs/2003.03327, 2020. URL <https://arxiv.org/abs/2003.03327>.